

Lab 11: MLOps with MLflow & Model Registry

Z5008 Big Data Lab · MTech Data Science & AI

Dr. Innocent Nyalala
innocent@iitmz.ac.in

School of Engineering & Science
IIT Madras Zanzibar

Monday, 25 May 2026
Assignment due: Sunday 31 May 2026, 23:59 EAT

Outline

The ML Project Reality

Common failure modes

- Notebook works; production crashes
- Cannot reproduce last month's model
- No record of which hyperparameters were used
- Multiple `model_v3_final_FINAL.pkl` files
- Data scientist leaves; knowledge gone
- Model degrades silently in production

**"It works on
my laptop!"**



3 months later:
"Which run
beat baseline?"



"The model
score dropped
— why?"

What is MLOps?

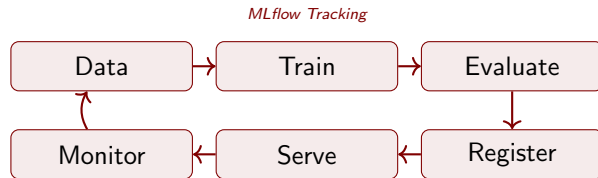
Definition

MLOps = **ML** + **DevOps** principles applied to the full ML lifecycle:

data → *training* → *evaluation* → *deployment*
→ *monitoring* → *retraining*

Core goals

- Reproducibility of every experiment
- Automation of training pipelines
- Reliable, versioned model deployment
- Continuous monitoring in production



MLOps Maturity Levels

Level	Description	Characteristics
0	Manual, script-driven	Jupyter notebooks, no automation, bespoke one-off deployment
1	ML pipeline automation	Automated training pipelines, Continuous Training (CT), feature store
2	CI/CD pipeline automation	Automated model build, test, deploy; full monitoring; A/B testing; auto-rollback

Where we are going

Level 0 → today's lab gets you to **Level 1**. Level 2 requires CI/CD integration (GitHub Actions, Jenkins)

MLflow: Four Core Components



Tracking

Log params, metrics, tags, artifacts per run



Projects

Package code + env for reproducible runs (MLproject)



Models

Standard format (flavors: sklearn, spark, pyfunc, xgboost)



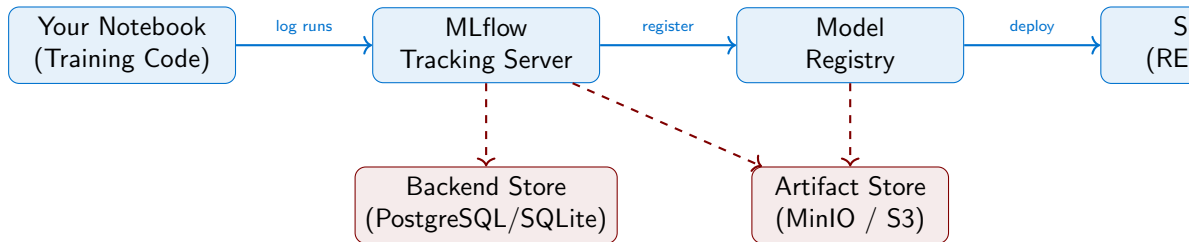
Registry

Central store with lifecycle stages

Our Lab 11 Docker environment

- MLflow tracking server: `http://mlflow:5000` (browser: `http://localhost:5000`)
- Artifact store: `s3a://mlflow-artifacts` (backed by MinIO at `http://minio:9000`)
- Backend store: PostgreSQL inside the mlflow container

The MLflow Architecture



Tracking Concepts

Hierarchy

Experiment $\supset$ **Run** $\supset$ {params, metrics, tags, artifacts}

- **Experiment:** logical grouping (e.g., “MobileMoney-Churn-v1”)
- **Run:** one training execution with unique `run_id`
- **Param:** key-value, set once (hyperparameter)
- **Metric:** numeric, optionally stepped (loss curves)
- **Tag:** free-form label (`git_hash`,

Golden rule

Log *everything* that affects reproducibility: data path, random seed, preprocessing choices, library versions.

Never hard-code run IDs

Always retrieve runs programmatically via `MlflowClient` or model registry URIs.

Setting Tracking URI & Experiment

```
1 import mlflow
2
3 # Point to the MLflow server running in Docker
4 mlflow.set_tracking_uri("http://mlflow:5000")
5
6 # Create (or retrieve) an experiment
7 mlflow.set_experiment("MobileMoney-Churn-v1")
8
9 # Verify
10 exp = mlflow.get_experiment_by_name("MobileMoney-Churn-v1")
11 print(f"Experiment ID : {exp.experiment_id}")
12 print(f"Artifact location: {exp.artifact_location}")
13 # Artifact location: s3a://mlflow-artifacts/1
```

Key note

`set_experiment()` creates the experiment if it does not exist. Subsequent calls with the same name are idempotent.

Starting a Run — Context Manager Pattern

```
1 with mlflow.start_run(run_name="lr-baseline") as run:
2     # --- log hyperparameters ---
3     mlflow.log_params({
4         "max_iter": 200,
5         "C": 1.0,
6         "solver": "lbfgs",
7         "random_state": 42,
8     })
9     # --- train ---
10    model = LogisticRegression(max_iter=200, C=1.0, random_state=42)
11    model.fit(X_train, y_train)
12    # --- log evaluation metrics ---
13    mlflow.log_metrics({
14        "train_accuracy": model.score(X_train, y_train),
15        "val_accuracy": model.score(X_val, y_val),
16        "val_f1": f1_score(y_val, model.predict(X_val)),
17    })
18    # --- optional free-form tag ---
19    mlflow.set_tag("data_version", "2026-05-snapshot")
```

Logging Model Artifacts with Signatures

```
1 from mlflow.models import infer_signature
2
3 with mlflow.start_run(run_name="rf-tuned") as run:
4     mlflow.log_params({"n_estimators": 300, "max_depth": 10,
5                       "random_state": 42})
6
7     rf = RandomForestClassifier(n_estimators=300, max_depth=10,
8                               random_state=42)
9     rf.fit(X_train, y_train)
10
11     # Infer schema of inputs and outputs
12     signature = infer_signature(
13         X_val,                # sample input DataFrame / ndarray
14         rf.predict(X_val)     # sample output
15     )
16     # Log the sklearn model with signature + small input example
17     mlflow.sklearn.log_model(
18         sk_model=rf,
19         artifact_path="model",
```

Auto-logging — sklearn

```
1 # Enable autolog BEFORE fitting
2 mlflow.sklearn.autolog(
3     log_input_examples=True,
4     log_model_signatures=True,
5     log_post_training_metrics=True,
6 )
7
8 with mlflow.start_run(run_name="rf-autolog"):
9     rf = RandomForestClassifier(n_estimators=200, random_state=42)
10    rf.fit(X_train, y_train)
11    # MLflow automatically logs:
12    #   params : n_estimators, max_depth, criterion, ...
13    #   metrics : training_accuracy_score, val_accuracy_score
14    #   artifact: model/ directory containing MLmodel file,
15    #               feature importances plot, confusion matrix
```

What autolog captures

All constructor params, fit duration, post-training metrics, model artifact, feature importances

Auto-logging — Spark MLlib

```
1 from pyspark.ml import Pipeline
2 from pyspark.ml.classification import GBTClassifier
3 from pyspark.ml.feature import VectorAssembler, StringIndexer
4
5 # Enable Spark autolog
6 mlflow.spark.autolog()
7
8 with mlflow.start_run(run_name="gbt-spark-autolog"):
9     indexer = StringIndexer(inputCol="label_str", outputCol="label")
10    assembler = VectorAssembler(inputCols=feature_cols,
11                                outputCol="features")
12    gbt = GBTClassifier(maxIter=20, maxDepth=5, seed=42)
13    pipeline = Pipeline(stages=[indexer, assembler, gbt])
14
15    model = pipeline.fit(train_df)
16    # MLflow logs: Pipeline params, training time,
17    #               and the full PipelineModel artifact
18    preds = model.transform(val_df)
19    acc = evaluator.evaluate(preds)
```

Logging Step-Wise Metrics (Training Curves)

```
1 from sklearn.linear_model import SGDClassifier
2
3 with mlflow.start_run(run_name="sgd-curves"):
4     mlflow.log_param("epochs", 50)
5     clf = SGDClassifier(max_iter=1, warm_start=True, random_state=42)
6
7     for epoch in range(1, 51):
8         clf.fit(X_train, y_train)          # one additional epoch
9         train_acc = clf.score(X_train, y_train)
10        val_acc    = clf.score(X_val, y_val)
11        # step= parameter makes metrics a time series in the UI
12        mlflow.log_metric("train_accuracy", train_acc, step=epoch)
13        mlflow.log_metric("val_accuracy", val_acc, step=epoch)
14
15    mlflow.sklearn.log_model(clf, "model")
```

MLflow UI shows these as interactive line charts

Navigate to the run → Metrics tab to view loss/accuracy curves over epochs.

Logging Custom Artifacts (Plots, Files)

```
1 import matplotlib.pyplot as plt
2 from sklearn.metrics import ConfusionMatrixDisplay
3 import os, tempfile
4
5 with mlflow.start_run(run_name="rf-with-artifacts"):
6     rf = RandomForestClassifier(n_estimators=100, random_state=42)
7     rf.fit(X_train, y_train)
8
9     with tempfile.TemporaryDirectory() as tmpdir:
10         # Confusion matrix plot
11         fig, ax = plt.subplots(figsize=(5, 4))
12         ConfusionMatrixDisplay.from_predictions(
13             y_val, rf.predict(X_val), ax=ax)
14         plot_path = os.path.join(tmpdir, "confusion_matrix.png")
15         fig.savefig(plot_path, bbox_inches="tight")
16         plt.close(fig)
17
18         # Log a single file under the "plots" sub-folder
19         mlflow.log_artifact(plot_path, artifact_path="plots")
```

Experiments tab

Lists all experiments; click one to see its runs.

Runs list

Table view: filter by metric, tag, param.

Sort by val_accuracy descending to find the best run.

Run detail page

- **Overview:** tags, git commit, start time
- **Parameters:** all logged key-values
- **Metrics:** numbers + step-wise plots
- **Artifacts:** file browser; preview images,

Comparing runs

Select ≥ 2 runs $\rightarrow$ click “Compare” $\rightarrow$ parallel coordinates plot, scatter plot, bar chart of chosen metrics.

Access in our lab

Browser: <http://localhost:5000>
(Docker maps container port 5000 to host 5000)

MLflow UI — Artifact Browser & Model Download

Artifact paths logged

- `model/` — MLmodel, conda.yaml, model binary
- `plots/confusion_matrix.png`
- `eval/` — any extra evaluation files

MLmodel file contents

Declares all available *flavors* so MLflow knows how to load the model (sklearn, python_function, etc.) and records the signature.

Register from the UI

On the run detail page → Artifacts → click `model/` → button “Register Model” appears in the top-right. Choose an existing registered model or create a new one.

Spark Session with S3A (MinIO)

```
1 from pyspark.sql import SparkSession
2
3 spark = (SparkSession.builder
4         .appName("Lab11-ML0ps")
5         .master("spark://spark-master:7077")
6         .config("spark.hadoop.fs.s3a.endpoint",
7               "http://minio:9000")
8         .config("spark.hadoop.fs.s3a.access.key", "minioadmin")
9         .config("spark.hadoop.fs.s3a.secret.key", "minioadmin")
10        .config("spark.hadoop.fs.s3a.path.style.access", "true")
11        .config("spark.hadoop.fs.s3a.impl",
12              "org.apache.hadoop.fs.s3a.S3AFileSystem")
13        .config("spark.jars.packages",
14              "org.apache.hadoop:hadoop-aws:3.3.4,"
15              "com.amazonaws:aws-java-sdk-bundle:1.12.319")
16        .getOrCreate())
17
18 mlflow.set_tracking_uri("http://mlflow:5000")
```

Logging a Full Spark MLlib Pipeline

```
1 mlflow.set_experiment("MobileMoney-Churn-v1")
2 mlflow.spark.autolog()
3
4 with mlflow.start_run(run_name="gbt-spark-pipeline") as run:
5     indexer    = StringIndexer(inputCol="label_str",
6                                outputCol="label")
7     assembler  = VectorAssembler(inputCols=feature_cols,
8                                outputCol="features")
9     gbt        = GBTClassifier(maxIter=20, maxDepth=5, seed=42)
10    pipeline   = Pipeline(stages=[indexer, assembler, gbt])
11
12    model      = pipeline.fit(train_df)
13
14    # Add metrics on top of what autolog already captured
15    from pyspark.ml.evaluation import BinaryClassificationEvaluator
16    evaluator   = BinaryClassificationEvaluator(
17        labelCol="label", metricName="areaUnderROC")
18    auc        = evaluator.evaluate(model.transform(val_df))
19    mlflow.log_metric("val_auc", auc)
```

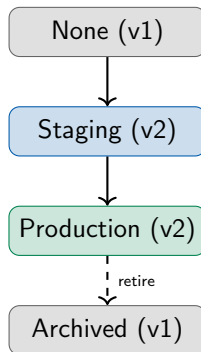
Model Registry Concepts

Registered Model

A named entity in the central registry. Multiple **versions** can exist simultaneously. Each version corresponds to one MLflow run artifact.

Version stages

- **None** — freshly registered
- **Staging** — under testing / shadow scoring
- **Production** — serving live traffic
- **Archived** — retired; still available



Registering a Model

```
1 from mlflow.tracking import MlflowClient
2 client = MlflowClient(tracking_uri="http://mlflow:5000")
3
4 # ---- Option 1: register at log time -----
5 with mlflow.start_run(run_name="rf-final") as run:
6     mlflow.sklearn.log_model(
7         sk_model=rf,
8         artifact_path="model",
9         registered_model_name="ChurnPredictor", # auto-registers
10        signature=signature,
11    )
12
13 # ---- Option 2: register an existing run after the fact -----
14 result = mlflow.register_model(
15     model_uri=f"runs://{run.info.run_id}/model",
16     name="ChurnPredictor",
17 )
18 print(f"Registered version: {result.version}") # e.g. 1
```

Managing Versions with MlflowClient

```
1 client = MlflowClient(tracking_uri="http://mlflow:5000")
2
3 # Add a human-readable description to the registered model
4 client.update_registered_model(
5     name="ChurnPredictor",
6     description="Binary churn classifier for East Africa mobile money.")
7
8 # Tag and describe a specific version
9 client.update_model_version(
10     name="ChurnPredictor", version="2",
11     description="RandomForest 300 trees -- beat baseline by +4% F1")
12 client.set_model_version_tag(
13     name="ChurnPredictor", version="2",
14     key="validated_by", value="Dr. Nyalala")
15
16 # Transition version 2 to Staging
17 client.transition_model_version_stage(
18     name="ChurnPredictor", version="2",
19     stage="Staging",
```

Promoting to Production

```
1 # Promote to Production; archive the current Production version
2 client.transition_model_version_stage(
3     name="ChurnPredictor",
4     version="2",
5     stage="Production",
6     archive_existing_versions=True,    # moves old Production to Archived
7 )
8
9 # Verify what is in Production
10 prod_versions = client.get_latest_versions(
11     name="ChurnPredictor", stages=["Production"])
12 for v in prod_versions:
13     print(f"    Production: version={v.version}    run_id={v.run_id}")
```

Model URI patterns

- By stage: models:/ChurnPredictor/Production
- By version: models:/ChurnPredictor/2

Loading a Model from the Registry

```
1 import mlflow.sklearn, mlflow.pyfunc, mlflow.spark
2
3 # -- sklearn flavor (original type is returned) --
4 sk_model = mlflow.sklearn.load_model(
5     "models:/ChurnPredictor/Production")
6 preds = sk_model.predict(X_new)          # ndarray
7
8 # -- pyfunc flavor (works for any framework) --
9 pyfunc_model = mlflow.pyfunc.load_model(
10    "models:/ChurnPredictor/Production")
11 preds_df = pyfunc_model.predict(X_new)    # DataFrame or ndarray
12
13 # -- Spark PipelineModel from registry --
14 spark_model = mlflow.spark.load_model(
15    "models:/ChurnPipelineSpark/Production")
16 result_df = spark_model.transform(new_spark_df)
17 result_df.select("prediction", "probability").show(5)
```


MLflow Projects — MLproject File

Listing 1: MLproject

```
name: churn-training

python_env: python_env.yaml

entry_points:
  main:
    parameters:
      n_estimators:
        type: int
        default: 100
      max_depth:
        type: int
        default: 6
      data_path:
        type: str
    command: >
      python train.py
```

What this enables

- Anyone can run: `mlflow run .`
- Parameters overridable from CLI
- Environment pinned via `python_env.yaml` or Docker image
- Each run is fully reproducible

Run from GitHub

```
mlflow run
https://github.com/org/repo
-P n_estimators=300
```

Serving via REST API

```
# Serve the Production model on port 1234 (run in a terminal)
mlflow models serve \
  --model-uri "models:/ChurnPredictor/Production" \
  --port 1234 \
  --no-conda

# Query with curl (split-orient JSON format)
curl -s -X POST http://localhost:1234/invocations \
  -H "Content-Type: application/json" \
  -d '{
    "dataframe_split": {
      "columns": ["age", "balance", "num_transactions", "days_active"],
      "data": [[32, 1200.5, 48, 180],
               [25, 400.0, 5, 30]]
    }
  }' | python -m json.tool
# {"predictions": [0, 1]} (0=no churn, 1=churn)
```

Scoring via Python requests

```
1 import requests, json
2
3 payload = {
4     "dataframe_split": {
5         "columns": ["age", "balance",
6                     "num_transactions", "days_active"],
7         "data": [
8             [32, 1200.5, 48, 180],
9             [25, 400.0, 5, 30],
10            [41, 3100.0, 120, 365],
11        ],
12    }
13 }
14
15 resp = requests.post(
16     "http://localhost:1234/invocations",
17     headers={"Content-Type": "application/json"},
18     data=json.dumps(payload),
19     timeout=10,
```

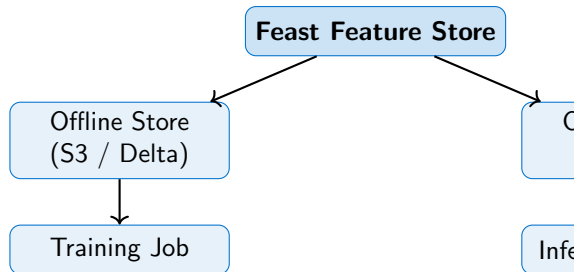
Feature Stores — Motivation & Feast

Problem without a feature store

- Training uses historical features; serving recomputes them differently → **training-serving skew**
- Same features recomputed by multiple teams
- No centralised feature discovery or governance

Feast key concepts

- **Entity**: object with an ID (`user_id`)
- **Feature View**: features from one data source



Tagging Git Hash & Data Snapshots

```
1 import subprocess, mlflow
2
3 def get_git_hash() -> str:
4     try:
5         return subprocess.check_output(
6             ["git", "rev-parse", "HEAD"]).decode().strip()
7     except Exception:
8         return "unknown"
9
10 with mlflow.start_run(run_name="reproducible-run"):
11     # Code version
12     mlflow.set_tag("git_hash", get_git_hash())
13     # Data snapshot (Delta table version or dated S3 path)
14     mlflow.set_tag("data_snapshot",
15                    "s3a://data/churn/20260520/")
16     mlflow.set_tag("data_version", "delta_v42")
17     # Environment
18     import sys, platform
19     mlflow.set_tag("python_version", sys.version.split()[0])
```

Data Drift & Concept Drift

Data drift (covariate shift)

Distribution of **input features** changes after deployment.

Example: mobile money transaction amounts shift after a new mobile product launches.

Concept drift

Relationship between inputs and target changes.

Example: user churn patterns shift during an economic downturn in East Africa.

Detection methods

- **PSI:** Population Stability Index; $PSI > 0.2$ signals major drift
- **KS test:** two-sample Kolmogorov-Smirnov for continuous features
- **Chi-squared test:** categorical features
- **ADWIN:** adaptive windowing for streaming drift detection
- **Evidently AI:** OSS monitoring library with HTML drift reports

Computing PSI in Python

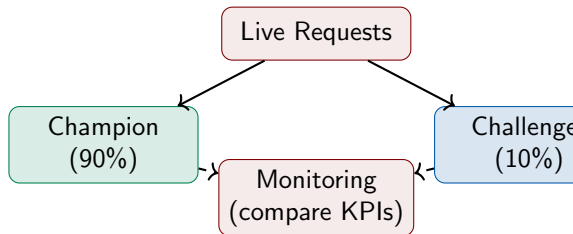
```
1 import numpy as np
2
3 def compute_psi(expected: np.ndarray, actual: np.ndarray,
4                 n_bins: int = 10) -> float:
5     """
6     Population Stability Index.
7     PSI < 0.1 : stable
8     0.1 -- 0.2 : moderate drift -- investigate
9     PSI > 0.2 : major drift -- retrain
10    """
11    bp = np.percentile(expected, np.linspace(0, 100, n_bins + 1))
12    exp_pct = np.histogram(expected, bins=bp)[0] / len(expected)
13    act_pct = np.histogram(actual, bins=bp)[0] / len(actual)
14    # Avoid log(0)
15    exp_pct = np.where(exp_pct == 0, 1e-4, exp_pct)
16    act_pct = np.where(act_pct == 0, 1e-4, act_pct)
17    psi = float(np.sum((act_pct - exp_pct) *
18                      np.log(act_pct / exp_pct)))
19    return psi
```

Champion-Challenger Deployment

Pattern

Route a fraction (e.g., 10%) of live traffic to the **challenger** model while the **champion** handles the remainder. Compare KPIs over time in production.

- $\text{Challenger} \geq \text{champion} + \text{threshold}$: promote challenger
- Challenger underperforms: automatic rollback
- Requires: request routing layer (NGINX, Kubernetes Ingress, Seldon)



CI/CD for ML — GitHub Actions Sketch

```
# .github/workflows/ml_retrain.yml
name: Scheduled Retraining

on:
  schedule:
    - cron: "0 2 * * 0"      # Every Sunday 02:00 UTC
  workflow_dispatch:        # Manual trigger

jobs:
  retrain:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.10" }
      - run: pip install -r requirements.txt
      - name: Train and log to MLflow
        env:
          MLFLOW_TRACKING_URI: ${ secrets.MLFLOW_URI }
```

Lab 11 Tasks — Workflow Summary

Today's hands-on sequence

- ➊ **Environment:** verify MLflow UI at localhost:5000; configure tracking URI in notebook
- ➋ **Multi-run tracking:** train ≥ 3 model variants (LR, RF, GBT), log all params / metrics / artifacts
- ➌ **Model Registry:** register the best run, transition Staging $\rightarrow$ Production, add descriptions and tags
- ➍ **Load & score:** load Production model from registry, run batch inference on unseen data
- ➎ **Serve:** mlflow models serve, send scoring request via curl or requests
- ➏ **Reflect:** drift detection, champion-challenger, MLOps maturity levels

Assignment due

Sunday 31 May 2026, 23:59 EAT — submit notebook + PDF screenshots to course portal.

Lab 11 — Docker Services Reference

Service	Container hostname	Port
JupyterLab	jupyter	8888
Spark Master UI	spark-master	8080
Spark Master RPC	spark-master	7077
Spark Worker UI	spark-worker	8081
Spark App UI	jupyter	4040
MLflow Tracking	mlflow	5000
MinIO Console	minio	9001
MinIO S3 API	minio	9000

Credentials

MinIO: minioadmin / minioadmin MLflow artifact bucket: mlflow-artifacts

Key Takeaways

Topics covered

- Why MLOps matters; maturity levels 0–2
- MLflow four components
- Tracking API: params, metrics, tags, artifacts
- Auto-logging: sklearn, Spark MLlib
- Model signatures for schema enforcement
- MLflow UI: run comparison, metrics plots
- Model Registry: stages, MlflowClient
- REST model serving & Python scoring

Five golden rules

- 1 Always set `tracking_uri` at notebook top
- 2 Log **everything**: seed, data version, git hash
- 3 Never hard-code run IDs; use `models:/Name/Stage`
- 4 Validate in **Staging** before promoting to **Production**
- 5 Monitor production for drift; retrain proactively

Next week — Lab 12

References & Further Reading

- MLflow documentation: mlflow.org/docs
- *Designing Machine Learning Systems* — Chip Huyen (O'Reilly, 2022)
- *Practical MLOps* — Noah Gift & Alfredo Deza (O'Reilly, 2021)
- Feast feature store: docs.feast.dev
- Evidently AI (drift monitoring): docs.evidentlyai.com
- Google MLOps whitepaper: cloud.google.com/resources/mlops-whitepaper



innocent@iitmz.ac.in



Room 105, Engineering Building, IIT Madras Zanzibar